

Robuster Kernelcode und Community-Workflow

Vom Bus-/Treiberpfad zum reviewfähigen Patch

5.05

Drei Fragen führen von Code zu Review

01

Ist der Pfad korrekt?

Bus, Device, Driver und Lebensdauer technisch nachvollziehen.

02

Welche Checks greifen?

Compiler, statische Analyse und Tests liefern unterschiedliche Evidenz.

03

Ist der Patch reviewfähig?

Scope, Begründung, Tests und Empfänger müssen nachvollziehbar sein.

Lernziel

Am Ende kannst du eine Treiberänderung technisch prüfen und reviewfähig vorbereiten.

Vier Übergänge machen Kernelcode robust

Verstehen

Aufrufkontext, Daten, Ownership und Lebensdauer klären.

Implementieren

Generische APIs nutzen und Fehler vollständig weitergeben.

Prüfen

Build, statische Analyse und Laufzeittest kombinieren.

Einreichen

Kleinen Patch mit Begründung und Evidenz adressieren.

QUALITÄTSPFAD

Kontext: Aufrufpfad → Daten → Ownership

Evidenz: Build → Analyse → Laufzeittest

Review: Scope → Commit → Empfänger

Bus, Device und Driver bilden den Prüfpfad

KERNSTRUKTUR

```
struct bus_type
struct device
struct device_driver
match() / probe()
remove() / devres
```

Was der Pfad verbindet

01 Beschreibung

Das Device hält Identität, Ressourcen und Zustand.

02 Auswahl

Der Bus vergleicht Device und Driver über match().

03 Lebensdauer

probe() baut auf; remove() und devres räumen zurück.

Registrierung, Matching und probe() führen zum Binding

QUELLPFAD

```
drivers/base/bus.c  
drivers/base/dd.c  
driver_register()  
driver_match_device()  
really_probe()
```

Binding in drei Schritten

01**Registrierung**

Treiber beim Bus bekannt machen

02**Matching**

IDs, Namen oder Firmwaredaten
vergleichen

03**Probe**

Ressourcen anfordern und Gerät
initialisieren

Ein erfolgreicher probe()-Pfad braucht einen symmetrischen Rückbau.

Fünf Dimensionen bestimmen Codequalität

QUALITÄT

Ownership

Kontext

Nebenläufigkeit

Fehlercodes

Portabilität

Zwei Sichtweisen

Lokale Korrektheit

Rückgabewerte, Grenzen und Fehlerpfade prüfen

Systemkorrektheit

Lebensdauer, Synchronisation und Schnittstellen verbinden

Code, der nur im Erfolgsfall funktioniert, ist noch nicht robust.

dev_err_probe() bewahrt Fehler und Kontext

TREIBER-CODE

```
irq = platform_get_irq(pdev, 0);  
if (irq < 0)  
    return dev_err_probe(dev, irq,  
                        "IRQ fehlt\n");
```

Lies als Vertrag

- 01 Fehlerwert**
Der ursprüngliche errno-Wert bleibt erhalten
- 02 Deferred Probe**
-EPROBE_DEFER wird passend behandelt
- 03 Kontext**
Die Meldung ist dem konkreten Device zugeordnet

APIs und empfohlene Muster immer gegen die verwendete Kernelversion prüfen.

Portabilität beginnt an Daten- und Hardwaregrenzen

PORTABILITÄT

```
fixed-width types  
__le32 / cpu_to_le32()  
readl() / writel()  
READ_ONCE() / WRITE_ONCE()  
IS_ENABLED(CONFIG_...)
```

Drei Grenzen

- 01 Datenformat**
Breite und Endianness explizit ausdrücken.
- 02 Hardware**
MMIO-APIs statt normaler Pointerzugriffe nutzen.
- 03 Varianten**
Konfigurations- und Architekturpfade mitbauen.

Werkzeuge prüfen unterschiedliche Fehlerklassen

PRÜFWERKZEUGE

```
make W=1  
make C=1  
checkpatch.pl  
Smatch / Coccinelle  
KUnit / selftests
```

Vom Hinweis zur Evidenz

01

Compiler

Typen, Warnungen und Konfigurationspfade prüfen.

02

Statisch

Adressräume, Datenfluss und Muster analysieren.

03

Dynamisch

Verhalten und Regression unter Laufzeit belegen.

Kein einzelnes Werkzeug beweist die Korrektheit einer Änderung.

Fehlerpfade sind Teil des Designs

RÜCKBAU

```
devm_*()  
goto err_disable  
cancel_work_sync()  
free_irq()  
put_device()
```

Zwei Ebenen

Reihenfolge

Ressourcen in umgekehrter Erwerbsreihenfolge freigeben.

Nebenläufigkeit

Asynchrone Arbeit stoppen, bevor Speicher verschwindet.

devm_*() vereinfacht Ownership, ersetzt aber keine Synchronisation.

Build, Laufzeit und Regression bilden den Prüfkreis

VERIFIKATION

```
allmodconfig  
W=1 / C=1  
lockdep / KASAN  
KUnit / selftests  
dmesg
```

Kernelversion, Konfiguration, Hardware und Testschritte im Bericht festhalten.

Der Kreislauf

01 Buildmatrix

Betroffene Architekturen und Konfigurationen übersetzen.

02 Laufzeitevidenz

Logs, Traces und Debug-Checks auf Warnungen prüfen.

03 Regression

Vorhandene und neue Tests reproduzierbar ausführen.

Ein guter Patch erklärt eine technische Entscheidung

PATCH-WORKFLOW

```
git diff --check
scripts/checkpatch.pl
git format-patch -1
scripts/get_maintainer.pl
git send-email --dry-run
```

Drei Phasen

- 01 Scope**
Ein Patch verfolgt einen klaren technischen Grund.
- 02 Commit**
Warum, Auswirkung und Tests nachvollziehbar beschreiben.
- 03 Empfänger**
Maintainer, Listen und Subsystemregeln korrekt wählen.

Praxis: Einen Patch technisch vorprüfen

TERMINAL

```
$ make W=1 C=1 M=drivers/...  
$ scripts/checkpatch.pl --strict 0001-*.patch  
$ scripts/get_maintainer.pl 0001-*.patch  
$ git diff --check
```

Belege drei Aussagen

- 01 Build**
Warnungen und betroffene Konfiguration dokumentieren
- 02 Semantik**
Static-Analysis-Befunde bewerten und begründen
- 03 Einreichung**
Committext, Maintainer und Testevidenz zusammenführen

Im Labor nichts versenden: Patch und Empfängerliste nur lokal als Dry Run prüfen.

Praxis: Treiberänderung reviewfähig machen

- | | | | |
|-----------|---|-----------|---|
| 01 | Änderung wählen
Eine kleine, klar begrenzte Treiberänderung auswählen. | 04 | Checks ausführen
Buildwarnungen, sparse und checkpatch bewerten. |
| 02 | Pfad verfolgen
Bus, match(), probe() und Ownership zuordnen. | 05 | Commit formulieren
Problem, technische Ursache, Änderung und Tests erklären. |
| 03 | Rückbau prüfen
Fehler- und remove()-Pfad gegen den Aufbau spiegeln. | 06 | Peer Review
Partner prüft Scope, Evidenz und Empfängerliste. |

Patch lokal behalten; keine Nachricht oder Änderung an externe Repositories senden.

Robuster Kernelcode verbindet Ownership, Evidenz und Review.

Formuliere in einem Satz:

Welche drei Belege machen deine Änderung technisch reviewfähig?